# Assignment 1: MCP Fundamentals

## Theory Questions

### 1. Explain the difference between a tool, a resource, and a prompt in MCP, with a one-line example of when you'd reach for each.

**Answer:**

In MCP, tools, resources, and prompts are used for different purposes.

- **Tool:** Used to perform an action or get something done.

  *Example:* Use a tool when Claude needs to send an email or calculate something.

- **Resource:** Used to provide information that Claude can read.

  *Example:* Use a resource when Claude needs to read a file or system information.

- **Prompt:** A ready-made instruction that helps guide Claude's response.

  *Example:* Use a prompt when you want Claude to summarize a document in a specific way.

**In short:**

| Component | Purpose | Example |
|---|---|---|
| Tool | Does an action | Send an email |
| Resource | Gives information | Read a file |
| Prompt | Gives instructions | Summarize a document |

### 2. A local MCP server usually talks to Claude Desktop over stdio instead of a normal HTTP API. What does that mean in practice, and why does it make sense for this use case?

**Answer:**

STDIO means that Claude Desktop starts the MCP server on the same computer and communicates with it through standard input and output.

A simple way to understand it is:

```
Claude Desktop
     |
     | stdin / stdout
     v
Local MCP Server
```

This is useful for a local MCP server because:

- No network setup is needed.
- The server can access local files and system resources when allowed.
- We do not need to open an HTTP port.

- It is simple for applications like Claude Desktop.

So, STDIO makes sense when both the MCP client and server are running on the same computer.

# FastMCP Documentation Questions

## 1. FastMCP supports three transports: stdio, http, and sse. Read the deployment docs and explain when you'd pick stdio vs http, and what actually changes in your mcp.run() call to switch between them.

**Answer:**

The transport depends on how the MCP server will be used.

**STDIO:**

I would use STDIO when the server is running locally and is started by a client such as Claude Desktop.

The normal call is:

```
mcp.run()
```

It can also be written as:

```
mcp.run(transport="stdio")
```

**HTTP:**

I would use HTTP when the server needs to be accessed through a network or used as a remote service.

For example:

```
mcp.run(
    transport="http",
    host="127.0.0.1",
    port=8000
)
```

The main difference is the transport setting. STDIO uses the input and output of the local process, while HTTP starts a server that clients can connect to.

| Transport | Use | Example |
|---|---|---|
| STDIO | Local server | `mcp.run()` |
| HTTP | Network or remote server | `mcp.run(transport="http")` |
| SSE | Older client compatibility | `mcp.run(transport="sse")` |

**2. FastMCP automatically turns some tool return values into "structured output" and others into plain text. Read the tools docs and explain the difference between what happens when a tool returns a plain string vs when it returns a dict or a Pydantic model, and why that distinction would matter to a client calling your tool.**

**Answer:**

FastMCP can return either simple text or structured data.

When a tool returns a **string**, the result is treated as normal text.

For example:

```
@mcp.tool
def greet() -> str:
    return "Hello, Tanishka!"
```

This is useful when the result is mainly meant to be read by a person.

When a tool returns a **dictionary**, the result can be handled as structured data.

For example:

```
@mcp.tool
def get_user() -> dict:
    return {
        "name": "Tanishka",
        "age": 19

    }
```

A **Pydantic model** can also be used when we want the returned data to have a fixed structure.

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

@mcp.tool
def get_user() -> User:
    return User(name="Tanishka", age=21)
```

The main difference is that plain text is easy for a person to read, while structured data is easier for another program to use. For example, a client can directly access the `name` and `age` fields instead of trying to extract them from a sentence.

**In short:**

| Return Value | Best Used For |
|---|---|
| String | Simple, human-readable response |
| Dictionary | Data that a program needs to process |
| Pydantic model | Structured data with validation |

So, structured output is useful when the client needs to work with individual values instead of treating the whole response as plain text.